

VisionInspect AI

Manufacturing Defect Detection & Quality Inspection System

Milestone 1 Report

Exploratory Data Analysis, Dataset Preprocessing & Model Configuration

Prepared by: Sarang Soni

Team: 4

Program: Infosys Springboard Internship 7.0

Date: July 2026

Table of Contents

1. Project Description	1
2. Dataset Description	1
3. Environment Setup	2
4. Methodology / Workflow	3
5. Implementation.....	4
5.1 Library Import & Dataset Verification	4
5.2 Dataset Exploration.....	4
5.3 Statistical Analysis.....	5
5.4 Data Visualization	6
5.5 Sample Image Display & Inspection	6
5.6 Image Preprocessing	7
5.7 Label Encoding & Dataset Preparation	8
6. Results & Observations	8
7. Key Observations	9
8. Conclusion	9
9. Future Work	10
10. References.....	10

1. Project Description

VisionInspect AI is an AI-powered manufacturing defect detection and quality inspection system that automates the identification of defective industrial products using computer vision and anomaly detection techniques. The goal of the project is to build a pipeline that can learn what a normal, defect-free product looks like and then automatically flag products that deviate from this norm, without requiring a large labelled dataset of every possible defect type.

The project is being developed using the PatchCore anomaly detection algorithm from the Anomalib framework, which learns feature representations of normal images and detects anomalies by comparing new images against this learned representation. The MVTec Anomaly Detection (MVTec AD) dataset is used for development and validation.

The objectives for this milestone were to:

- Set up the development environment and project structure.
- Explore and understand the structure and composition of the MVTec AD dataset.
- Perform statistical analysis and visualization of the dataset (Exploratory Data Analysis).
- Build and verify an image preprocessing pipeline (resizing, colour conversion, normalization).
- Prepare labelled, preprocessed data ready for model training in the next milestone.

2. Dataset Description

2.1 Dataset Overview

The project uses the MVTec Anomaly Detection (MVTec AD) dataset, a widely used benchmark for industrial anomaly detection. It contains high-resolution RGB images across 15 object and texture categories: bottle, cable, capsule, carpet, grid, hazelnut, leather, metal nut, pill, screw, tile, toothbrush, transistor, wood, and zipper.

Each category is organized into train, test, and ground_truth folders. The train folder contains only normal (defect-free) images, which is what the PatchCore model is trained on. The test folder contains both normal (good) images and a range of category-specific defect types, and the ground_truth folder provides pixel-level masks that localize the defect region for every defective test image.

2.2 Dataset Structure

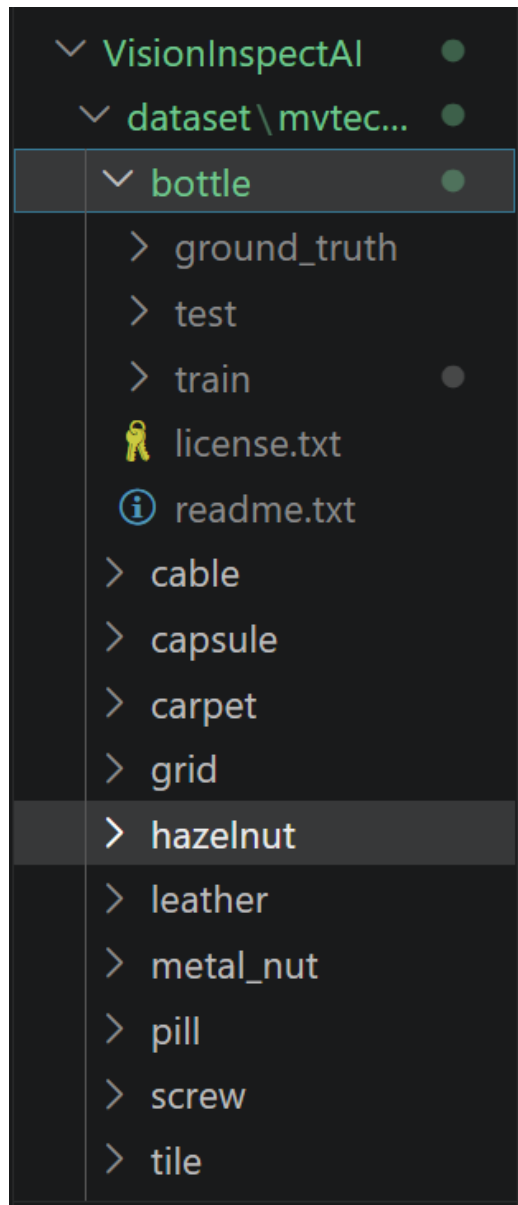
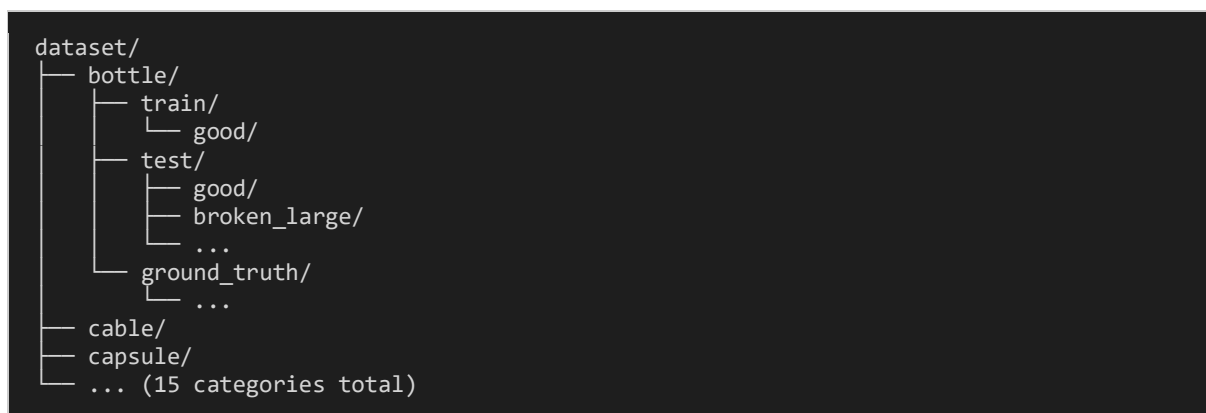


Fig 2.1: MVtec AD dataset folder structure (Visual Studio Code explorer view)



2.3 Dataset Statistics

A category-wise breakdown of the dataset was generated by counting the images in the train/good, test/good, and defect sub-folders of every category. The results are summarized below.

Category	Train (Good)	Test (Good)	Defect Images	Total Images
Bottle	209	20	63	292
Cable	224	58	92	374
Capsule	219	23	109	351
Carpet	280	28	89	397
Grid	264	21	57	342
Hazelnut	391	40	70	501
Leather	245	32	92	369
Metal Nut	220	22	93	335
Pill	267	26	141	434
Screw	320	41	119	480
Tile	230	33	84	347
Toothbrush	60	12	30	102
Transistor	213	60	40	313
Wood	247	19	60	326
Zipper	240	32	119	391

Table 2.1: Category-wise dataset statistics

```

DATASET SUMMARY
=====
Number of Categories: 15
Training Images: 3629
Testing Good Images: 467
Defect Images: 1258
Total Images: 5354

```

Output 2.1: Overall dataset summary generated from the notebook

3. Environment Setup

The AI module of VisionInspect AI was developed and executed locally using Jupyter Notebook within Visual Studio Code, with a dedicated Python virtual environment used to manage dependencies and avoid package conflicts.

3.1 Development Environment

- IDE: Visual Studio Code (Jupyter Notebook)
- Programming Language: Python
- Virtual Environment: Python venv (.venv)

- Version Control: Git & GitHub

3.2 Libraries Used

- OpenCV (cv2) — image loading, resizing, colour-space conversion
- NumPy — numerical operations on image arrays
- Pandas — tabulating and summarizing dataset statistics
- Matplotlib — visualization of dataset distribution and sample images
- Pillow (PIL) — supplementary image handling

The following libraries are planned for the next milestone, once model development begins: PyTorch, Anomalib, and Torchvision, which together provide the PatchCore implementation and pretrained backbone required for anomaly detection.

3.3 Project Structure

```
VisionInspectAI/
├── dataset/
│   └── mvtec_anomaly_detection/
├── output_charts/
├── src/
│   └── notebooks/
│       └── MVTec_AD_EDA.ipynb
├── README.md
├── requirements.txt
├── .gitignore
└── LICENSE
```

4. Methodology / Workflow

The notebook follows a sequential workflow that takes the raw dataset through verification, exploration, statistical analysis, visualization, and preprocessing before it is handed off to model training in the next milestone.

```
Dataset Path Verification
      |
      v
Category Listing --> Folder Structure Check
      |
      v
Image Counting (train / test / defect)
      |
      v
Statistical Summary (DataFrame + totals)
      |
      v
Visualization (category distribution bar chart)
      |
      v
Sample Image Display --> Image Inspection (shape, dtype, pixel range)
      |
      v
Preprocessing (resize -> BGR to RGB -> normalize)
      |
      v
```

Label Encoding --> Full Dataset Preparation --> Verification

Fig 4.1: End-to-end EDA and preprocessing workflow

5. Implementation

This section documents the modules implemented in the EDA notebook (MVTec_AD_EDA.ipynb), along with the corresponding code and generated output.

5.1 Library Import & Dataset Verification

The required libraries were imported, and the dataset path was verified before any further processing.

```
import os
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from PIL import Image
%matplotlib inline

dataset_path = r'D:\Internship_Project\VisionInspectAI\dataset\mvttec_anomaly_detection'
print('Dataset Exists:', os.path.exists(dataset_path))
```

Dataset Exists: True

5.2 Dataset Exploration

All 15 category folders were listed, and the internal folder structure of each category (train, test, ground_truth) was verified.

```
categories = sorted([
    f for f in os.listdir(dataset_path)
    if os.path.isdir(os.path.join(dataset_path, f))])
print(f'Total Categories: {len(categories)}')
for c in categories:
    print(c)
```

Total Categories: 15
 bottle cable capsule carpet grid hazelnut
 leather metal_nut pill screw tile toothbrush
 transistor wood zipper

```
for category in categories:
    category_path = os.path.join(dataset_path, category)
    print(f'\n {category}')
    for folder in os.listdir(category_path):
        print('  +--', folder)
```

bottle
 +-- ground_truth
 +-- license.txt
 +-- readme.txt
 +-- test
 +-- train

... (repeated for all 15 categories)

5.3 Statistical Analysis

For every category, the number of normal (good) training images, normal test images, and defective test images was counted, and the results were compiled into a pandas DataFrame.

```
dataset_info = []
for category in categories:
    cp = os.path.join(dataset_path, category)
    train_good = len(os.listdir(os.path.join(cp, 'train', 'good')))
    test_good = len(os.listdir(os.path.join(cp, 'test', 'good')))
    defect_images = 0
    for defect in os.listdir(os.path.join(cp, 'test')):
        if defect != 'good':
            defect_images += len(os.listdir(os.path.join(cp, 'test', defect)))
    total = train_good + test_good + defect_images
    dataset_info.append([category, train_good, test_good, defect_images, total])

df = pd.DataFrame(dataset_info,
                  columns=['Category', 'Train Good', 'Test Good', 'Defect Images', 'Total Images'])
df
```

	Category	Train Good	Test Good	Defect Images	Total Images
0	bottle	209	20	63	292
1	cable	224	58	92	374
2	capsule	219	23	109	351
3	carpet	280	28	89	397
4	grid	264	21	57	342
5	hazelnut	391	40	70	501
6	leather	245	32	92	369
7	metal_nut	220	22	93	335
8	pill	267	26	141	434
9	screw	320	41	119	480
10	tile	230	33	84	347
11	toothbrush	60	12	30	102
12	transistor	213	60	40	313
13	wood	247	19	60	326
14	zipper	240	32	119	391

Fig 5.1: Category-wise statistics DataFrame output (Jupyter Notebook)

An overall dataset summary was then printed by aggregating the DataFrame columns:

```
print('DATASET SUMMARY')
print('=' * 50)
print('Number of Categories:', len(df))
print('Training Images:', df['Train Good'].sum())
print('Testing Good Images:', df['Test Good'].sum())
print('Defect Images:', df['Defect Images'].sum())
print('Total Images:', df['Total Images'].sum())
```


5.4 Data Visualization

A bar chart was generated using Matplotlib to visualize the total number of images available per category.

```
plt.figure(figsize=(12,6))
plt.bar(df['Category'], df['Total Images'])
plt.xticks(rotation=45)
plt.xlabel('Category')
plt.ylabel('Number of Images')
plt.title('Total Images per Category')
plt.show()
```

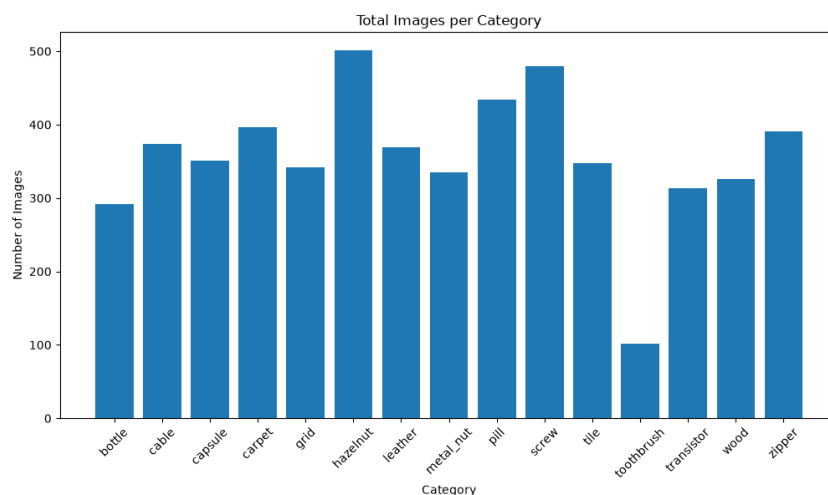


Fig 5.2: Total images per category

5.5 Sample Image Display & Inspection

A 3x5 grid of sample 'good' images — one per category — was plotted to visually confirm the dataset contents. A single sample image (bottle) was then loaded individually and inspected in detail.

```
fig, axes = plt.subplots(3, 5, figsize=(18,10))
axes = axes.ravel()
for i, category in enumerate(categories):
    folder = os.path.join(dataset_path, category, 'train', 'good')
    img = cv2.imread(os.path.join(folder, os.listdir(folder)[0]))
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    axes[i].imshow(img)
    axes[i].set_title(category)
    axes[i].axis('off')
plt.show()
```

```
print('='*50)
print('IMAGE INFORMATION')
print('='*50)
print('Shape      : ', image.shape)
print('Height     : ', image.shape[0])
print('Width      : ', image.shape[1])
print('Channels    : ', image.shape[2])
print('Data Type   : ', image.dtype)
print('Minimum Pixel : ', image.min())
print('Maximum Pixel : ', image.max())
```

```
=====
IMAGE INFORMATION
=====
Shape      : (900, 900, 3)
Height     : 900
Width      : 900
Channels    : 3
Data Type  : uint8
Minimum Pixel : 29
Maximum Pixel : 255
```

5.6 Image Preprocessing

Each image was resized to a fixed resolution of 256x256, converted from OpenCV's default BGR colour order to RGB, and normalized so that pixel values fall within the [0, 1] range required for downstream model training.

```
resized_image = cv2.resize(image, (256,256))
print('Original Shape :', image.shape)
print('Resized Shape :', resized_image.shape)

rgb_image = cv2.cvtColor(resized_image, cv2.COLOR_BGR2RGB)

normalized_image = rgb_image.astype(np.float32) / 255.0
print('Minimum Pixel :', normalized_image.min())
print('Maximum Pixel :', normalized_image.max())
print('Datatype :', normalized_image.dtype)
```

```
Original Shape : (900, 900, 3)
Resized Shape : (256, 256, 3)
Minimum Pixel : 0.1137
Maximum Pixel : 1.0
Datatype : float32
```

The original and preprocessed versions of the sample image were plotted side by side to visually confirm that image content was preserved through the pipeline.

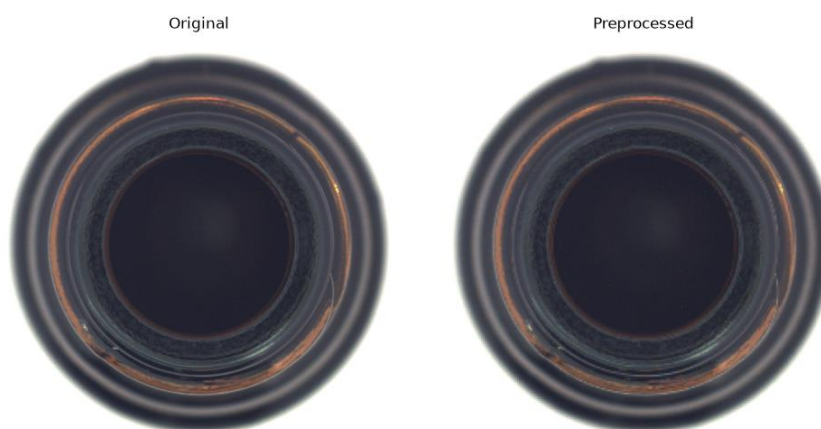


Fig 5.3: Original image vs. preprocessed image (bottle category)

5.7 Label Encoding & Dataset Preparation

Since PatchCore is trained only on normal (defect-free) images, a simple binary labelling scheme was defined — 0 for good images and 1 for defective images — for later use in evaluation. The full preprocessing pipeline (resize, RGB conversion, normalization) was then applied to every training image across all 15 categories.

```
label_good = 0
label_defect = 1

processed_images = []
labels = []
for category in categories:
    train_good = os.path.join(dataset_path, category, 'train', 'good')
    for file in os.listdir(train_good):
        img_path = os.path.join(train_good, file)
        img = cv2.imread(img_path)
        img = cv2.resize(img, (256,256))
        img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
        img = img.astype(np.float32) / 255.0
        processed_images.append(img)
        labels.append(0)

print('Total Training Images Processed :', len(processed_images))
print('Total Labels :', len(labels))
```

```
Good Images Label : 0
Defect Images Label : 1
Total Training Images Processed : 3629
Total Labels : 3629
```

A final verification step confirmed that every processed image had the expected shape, datatype, and pixel range prior to being used for model training.

```
print('Image Shape :', processed_images[0].shape)
print('Datatype :', processed_images[0].dtype)
print('Minimum Pixel :', processed_images[0].min())
print('Maximum Pixel :', processed_images[0].max())
print('Label :', labels[0])
```

```
Image Shape : (256, 256, 3)
Datatype : float32
Minimum Pixel : 0.11372549
Maximum Pixel : 1.0
Label : 0
```

6. Results & Observations

The EDA and preprocessing pipeline was successfully executed across all 15 categories of the MVTec AD dataset. The dataset was confirmed to contain 3,629 normal training images, 467 normal test images, and 1,258 defective test images, for a total of 5,354 images. Every image was successfully resized to a uniform 256x256x3 format, converted to RGB, and normalized to the [0, 1] range, with no corrupted or unreadable files encountered.

The category-wise distribution (Fig 5.2) shows that image counts vary noticeably across categories — hazelnut and screw have the largest number of images (501 and 480 respectively), while toothbrush

has the fewest (102). This imbalance is an important consideration for the model training stage in the next milestone.

7. Key Observations

- The dataset is organized consistently across all 15 categories, each with train, test, and ground_truth folders.
- A clear class imbalance exists between categories — toothbrush has roughly one-fifth the images of hazelnut.
- Preprocessing produced a uniform output shape of 256 x 256 x 3 across every image, making the dataset ready for model input.
- Pixel values were consistently normalized to a [0.0, 1.0] float32 range after preprocessing.
- No corrupted or unreadable images were found during exploration.
- Visual comparison of original vs. preprocessed images confirmed that resizing and normalization did not distort image content.

8. Conclusion

This milestone focused on building a solid data foundation for the VisionInspect AI project. The MVTec AD dataset was explored in detail, its structure and class distribution were verified, and a preprocessing pipeline was implemented and validated end-to-end — from raw images through resizing, colour conversion, and normalization, to a fully labelled set of 3,629 training images ready for model input.

These outputs establish a verified, reproducible starting point for the next phase of the project, which will focus on configuring and training the PatchCore anomaly detection model.

9. Future Work

- Complete configuration of the PatchCore model (backbone, feature layers, coreset sampling ratio).
- Extract deep feature embeddings using a pretrained Wide ResNet50-2 backbone.
- Train the model on normal images and build the feature memory bank.
- Evaluate the model and generate anomaly scores, heatmaps, and predicted defect masks.
- Extend the pipeline to a prediction module and, eventually, a web-based inspection dashboard.

10. References

1. MVTec AD Dataset — <https://www.mvtec.com/research-teaching/datasets/mvtec-ad>
2. Anomalib Documentation — <https://anomalib.readthedocs.io/>
3. Python Documentation — <https://docs.python.org/3/>
4. OpenCV Documentation — <https://docs.opencv.org/>

5. NumPy Documentation — <https://numpy.org/doc/>
6. Pandas Documentation — <https://pandas.pydata.org/docs/>
7. Matplotlib Documentation — <https://matplotlib.org/>